

Week 37 - Graded Mini Project

Learning Outcome Addressed

- Understand the principles of conditional generative modeling and how class labels can be incorporated into deep learning architectures.
- Learn the end-to-end pipeline for image generation using Conditional Generative Adversarial Networks (cGANs), including data loading, preprocessing, model design, training, and evaluation.
- Gain practical experience in designing and training generator and discriminator networks that jointly learn from image data and class labels.
- Develop skills to evaluate and visualise generated outputs, assessing model performance based on visual quality and label consistency.

Objective

- Understand the role of conditioning in generative modelling.
- Implement a generator and discriminator that use class label information.
- Train a cGAN to generate images based on labels (cat or dog).
- Visualise the generation process and assess output quality.

Submission Instructions

Please document your response on the following pages.

Once you have completed the activity, save the file as a PDF and upload it. Be sure to name the file as **Module 37: Graded Mini Project_[Your last name]**.

Your submission will be considered complete when it meets the following criteria:

- Includes all the key elements outlined in the activity instructions and the rubric.
- Adheres to the submission guidelines.
- Is submitted on time.

(Note: Kindly provide the output in Jupyter Notebook or Python script with all code and comments.)

This is a required activity and counts towards programme completion.

Reflect on the task and respond to the following questions.

Data Description

The Cats vs Dogs dataset contains over 23,000 labeled images. For this project:

- Label 0 = Cat
- Label 1 = Dog

You will use unpaired, labelled images to train your cGAN model. The dataset can be directly loaded using tensorflow_datasets.

Tasks:

Part 1: Data Preparation

1. Dataset Loading:

- Use TensorFlow Datasets (cats_vs_dogs)

Use below code snippet to load it-

```
import tensorflow as tf
import tensorflow_datasets as tfds
import matplotlib.pyplot as plt
import numpy as np
from tensorflow.keras import layers
(ds_train, _), ds_info = tfds.load(
    'cats_vs_dogs',
    split=['train[:90%]', 'train[90%:]'],
    shuffle_files=True,
    as_supervised=True,
    with_info=True
```

- Split into training and testing subsets

2. Preprocessing:

- Resize all images to 64×64 pixels
- Normalize pixel values to the [-1, 1] range
- Cast labels to integer format
- Create training batches with labels

Part 2: Conditional GAN Architecture

1. Generator:

- Accepts both noise vector and class label as input
- Uses label embedding and concatenates it with noise
- Builds up to a 64×64×3 image using Conv2DTranspose layers

2. Discriminator:

- Accepts both real/fake image and class label
- Embeds label and concatenates it with image
- Uses Conv2D layers to classify input as real/fake

Part 3: Loss Functions and Optimizer

- **Generator Loss:** Binary cross-entropy encouraging discriminator to classify generated images as real
- **Discriminator Loss:** Binary cross-entropy distinguishing real vs fake images
- **Optimizers:** Adam optimizer for both networks

Part 4: Training the Model

- Train the generator and discriminator alternately
 - Use `@tf.function` for optimized performance
 - Train for a minimum of 10 epochs
 - Print training progress per epoch
-

Part 5: Evaluation and Visualization

- Generate images conditioned on randomly chosen labels
 - Display generated images with corresponding label titles
 - Ensure images are normalized back to `[0, 1]` for display
-